

SOFTWARE REQUIREMENTS SPECIFICATION

SkillForge AI: Personalized Learning & Career Mentor

Prepared By: Chetan Dabhi

Role: Lead Architect

Target Environment: Hackathon Submission

Date: August 2026

1. Introduction

| 1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive overview of the SkillForge AI platform. It outlines the complete system architecture, functional modules, database schemas, and data workflows. This document serves as the absolute source of truth for the development team and autonomous AI agents contributing to the codebase during the hackathon.

1.2 Scope

SkillForge AI is a web-based educational technology platform designed to bridge the gap between a student's current skill set and industry expectations. The system utilizes Large Language Models (LLMs) to extract data from user resumes, perform a gap analysis against a designated "Dream Job," and generate a personalized, interactive learning roadmap. Furthermore, the platform augments this learning path by recommending targeted certifications, curating tiered portfolio projects, and providing specialized interview preparation.

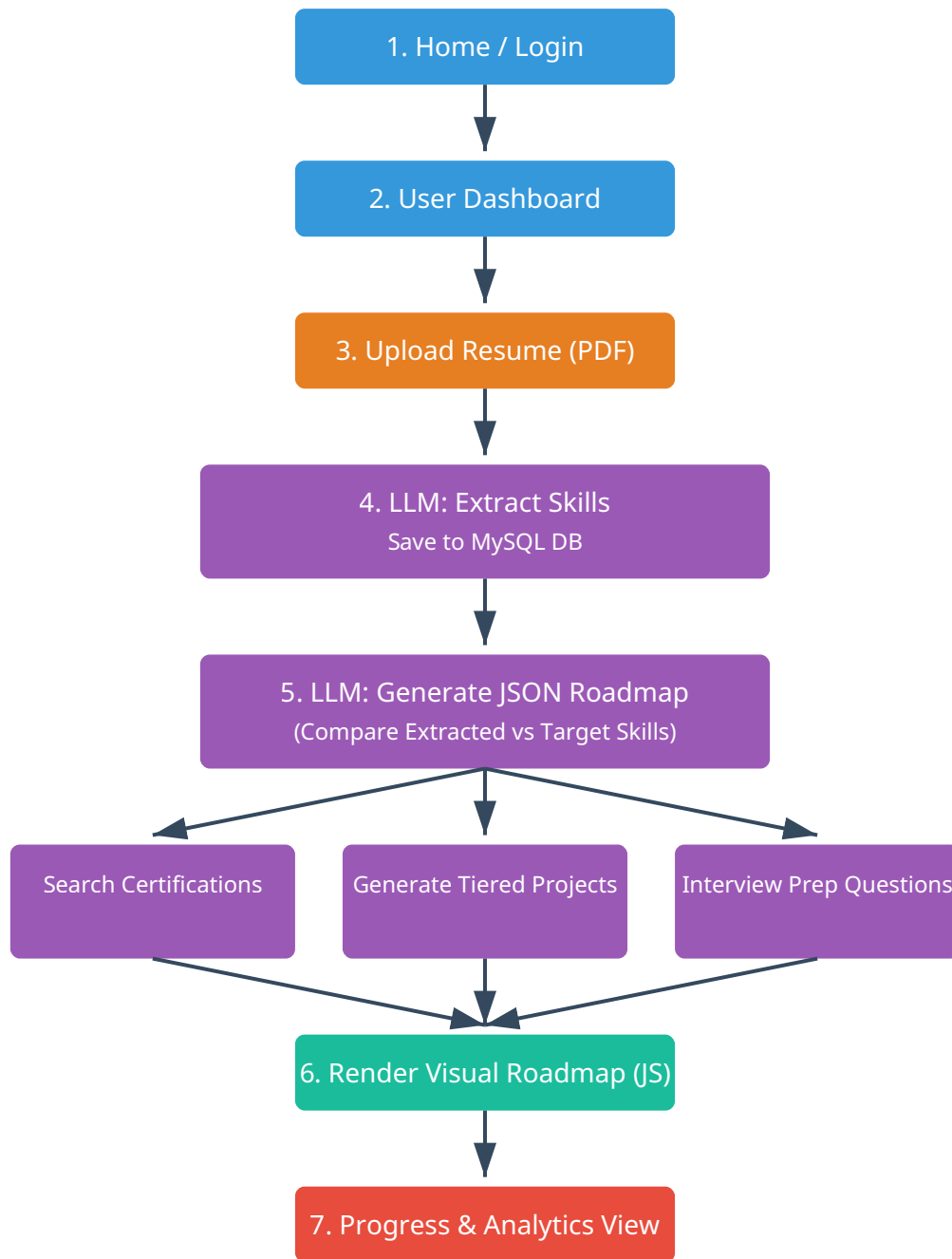
2. Technology Stack

To ensure rapid development, robust performance, and compliance with hackathon rules, the following technology stack has been strictly defined:

- **Frontend Layer:** HTML5, CSS3, and Vanilla JavaScript. JavaScript is exclusively responsible for parsing the JSON outputs from the AI and rendering the interactive visual roadmap.
- **Backend Engine & Orchestration:** Python (FastAPI or Flask). Python acts as the central router, handling file uploads (resumes), orchestrating prompt engineering, and managing RESTful endpoints.
- **Database Layer:** MySQL. A relational structure is enforced to handle user profiles, relationship mapping between roadmaps and projects, and the atomic tracking of completed roadmap nodes for analytics.
- **AI & Intelligence Layer:** Large Language Models API. Utilized for parsing unstructured text, generating structured JSON roadmaps, and synthesizing custom educational assets.

3. System Architecture & Flowchart

The system operates on a linear, pipeline-based architecture. Data flows from user ingestion to AI processing, culminating in the rendering of a dynamic, client-side interface.



4. Detailed Functional Requirements

This section details the internal logic and specific behaviors required for every module of the platform.

4.1 Phase 1: Onboarding & Data Extraction

Requirement 1.1: Authentication. The system must support basic local authentication. User sessions should be managed securely by the Python backend.

Requirement 1.2: Resume Ingestion. The Dashboard must include a drag-and-drop file uploader supporting `.pdf` files. The backend will utilize Python libraries to parse the raw text from the file.

Requirement 1.3: LLM Skill Extraction. The parsed text is sent to the LLM via a structured prompt.

```
System Prompt: "You are an expert technical recruiter. Read the following resume text and extract the candidate's core technical skills, programming languages, tools, and soft skills. Output the result strictly as a comma-separated list."
```

Persona Use Case

If a web development and computer engineering student with foundational knowledge in HTML, CSS, JavaScript, Python, and SQL uploads their resume, the LLM must accurately parse this data and write it to the MySQL database associated with their user ID. This forms the baseline for all subsequent generation.

4.2 Phase 2: Core Engine & Roadmap Generation

Requirement 2.1: Target Definition. The user inputs their "Dream Job" (e.g., "Red Team Security Analyst").

Requirement 2.2: Multi-Step Gap Analysis. The backend executes a chained prompt sequence:

1. **Industry Profiling:** Ask the LLM to generate the standard required skills for the Dream Job.
2. **Comparison Logic:** Provide the LLM with the user's extracted skills (from Phase 1) AND the industry requirements. Instruct the LLM to identify the delta (the missing skills).
3. **JSON Formatting:** Instruct the LLM to output a hierarchical learning roadmap covering the missing skills, ensuring strict compliance with a predefined JSON schema.

```
{
  "roadmap": {
    "goal": "Red Team Security Analyst",
    "phases": [
      {
        "phase_id": 1,
        "title": "Networking Fundamentals",
        "topics": ["TCP/IP", "Subnetting", "OSI Model"]
      }
    ]
  }
}
```

4.3 Phase 3: Post-Generation Enhancements

Once the JSON roadmap is validated, the Python backend executes three parallel sub-routines:

Requirement 3.1: Certification Generation. Based on the topics in the JSON roadmap, the LLM generates a list of 2-3 recognized industry certifications.

Requirement 3.2: Tiered Portfolio Projects. The LLM must synthesize projects categorized into three tiers:

- **Beginner:** Focuses on fundamentals. (e.g., "Develop a basic local Wi-Fi Intrusion Detection System using Python"). Must include a list of prerequisite skills.
- **Intermediate:** Combines multiple concepts.
- **Advanced:** Capstone-level project for resume inclusion.

Requirement 3.3: Interview Preparation. Generate 5 technical and 2 behavioral interview questions relevant to the Dream Job.

4.4 Phase 4 & 5: Visualization and Analytics

Requirement 4.1: Interactive Render. The frontend JavaScript retrieves the generated JSON and renders a visual node-based map. Users can click on nodes to view details and mark them as "Completed."

Requirement 5.1: Progress Calculation. Every time a user marks a node as completed, an asynchronous request is sent to the Python backend to update the database state. The Analytics Dashboard calculates the overall completion percentage: $(\text{Completed Nodes} / \text{Total Nodes}) * 100$.

5. Database Schema (MySQL)

The system requires a highly normalized database structure to efficiently track user progression and roadmap complexity.

Table: Users

Column	Type	Description
user_id	INT (PK)	Auto-incrementing primary key.
name	VARCHAR(100)	Full name of the user.
email	VARCHAR(100)	Unique login email.
password_hash	VARCHAR(255)	Securely hashed password.

Table: User_Profiles

Column	Type	Description
profile_id	INT (PK)	Auto-incrementing primary key.
user_id	INT (FK)	References Users(user_id).
extracted_skills	TEXT	Comma-separated list of parsed skills.
dream_job	VARCHAR(100)	Target goal input by user.

Table: Roadmaps

Column	Type	Description
roadmap_id	INT (PK)	Auto-incrementing primary key.
user_id	INT (FK)	References Users(user_id).
raw_json	JSON	The complete JSON output from the LLM.
generated_at	TIMESTAMP	Time of creation.

Table: Roadmap_Nodes (For Progress Tracking)

Column	Type	Description
node_id	INT (PK)	Auto-incrementing primary key.
roadmap_id	INT (FK)	References Roadmaps(roadmap_id).
topic_name	VARCHAR(150)	Name of the specific learning topic.
is_completed	BOOLEAN	Defaults to FALSE. Used for Analytics calculation.

6. Non-Functional Requirements

- **Performance:** The entire AI generation process (Resume Parsing -> Roadmap -> Projects -> Certifications) must complete within 15 seconds to ensure a seamless User Experience.
- **Usability:** The platform interface must be fully responsive, ensuring the visual roadmap is navigable on both desktop and mobile viewports.
- **Open Source Compliance:** Per Hackathon Rule 4, all Python libraries (e.g., Flask, PyPDF2) and frontend frameworks must be open-source or academic-tier free. Proprietary AI APIs must be declared in the final repository README.